

हेलो! आपका स्वागत है **Unit 12: Destructuring, Spread & Modern JS** की complete study class में.

मैंने आपके workspace में, unit 12 destructive folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter12_modern_js.md
- Code Examples File: modern_js_examples.js
- Practice Exercises File: modern_js_practice.js

चलिए अब Modern JS parameters, destructuring systems, spread operations, optional chains variables map features ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: DESTRUCTURING TECHNIQUES

Destructuring data parameters structures (Arrays aur Objects) se variables ko unpack extract karne ka shorthand layout syntax hai.

1. Array Destructuring

What is it? Array index positions elements values ko clean variables pointers array formatting brackets [] use karke variable map assign.

How does it work? Left side mein variables ka array format brackets specify karein:

- **Skipping values:** comma keys , spaces ignore. E.g. [a, , b].
- **Default values:** placeholder values assigns if missing elements inputs: [a = 1].

Simple example

javascript

```
let colors = ["red", "green", "blue"];
const [firstColor, , thirdColor, fourthColor = "yellow"] = colors;
console.log(firstColor);
console.log(thirdColor);
console.log(fourthColor);
```

Output red blue yellow

2. Object Destructuring

What is it? Object properties key variables matching names extract parameters systems checking values sets.

How does it work? Left side mein curly braces {} key names target write checks:

- **Renaming variables:** use alias renaming colon formatting: { key: newAlias }.
- **Default values:** placeholder properties: { key = defaultValue }.

Simple example

javascript

```
let user = { username: "ravi_dev", age: 24 };  
const { username: accountName, age, country = "India" } = user;  
console.log(accountName);  
console.log(country);
```

Output ravi_dev India

3. Destructuring with for...of

What is it? Objects inside array element loop variables lists unpack direct destructuring properties extraction.

Simple example

javascript

```
let students = [  
  { name: "Rahul", score: 95 },  
  { name: "Amit", score: 88 }  
];  
for (let { name, score } of students) {  
  console.log(`${name} scored ${score}`);  
}
```

Output Rahul scored 95 Amit scored 88

PART 2: UNPACKING & GATHERING VALUES

1. Spread Operator (...)

What is it? Spread operator array elements variables arrays objects parameters items data sets values **expand (khól deta hai)**.

Why is it needed? Safely copy arrays elements, merge parameters structures list keys.

Simple example

javascript

```
// Merge arrays
```

```
let a = [1, 2];
```

```
let b = [...a, 3, 4];
```

```
console.log(b);
```

```
// Merge objects
```

```
let car = { brand: "Honda" };
```

```
let updatedCar = { ...car, year: 2022 };
```

```
console.log(updatedCar);
```

Output [1, 2, 3, 4] { brand: "Honda", year: 2022 }

2. Rest Operator (...)

What is it? Function arguments inputs parameters variables dynamic rest parameters check array **collect (ektha kar leta hai)**.

Why is it needed? Unknown sizes arguments variables handle checklists arrays sets dynamically.

Simple example

javascript

```
function report(leader, ...team) {
```

```
    console.log("Leader: " + leader);
```

```
    console.log("Team Array:", team);
```

```
}
```

```
report("Aman", "Ravi", "Amit", "Rahul");
```

Output Leader: Aman Team Array: ["Ravi", "Amit", "Rahul"]

3. Spread vs Rest — Clear Difference

- **Spread Operator:** Expands/opens up values (converting arrays list elements to individual parameters). Used during assignments or merge calls.
- **Rest Operator:** Collects multiple single arguments inputs into a single array block. Used inside function parameter arguments declarations.

PART 3: MODERN ES6 HELPER UTILITIES

1. Template Literals

What is it? Backticks (`) formatting strings interpolation variables parameters matching `${expression}` syntax.

Simple example

javascript

```
let system = "MacBook";
```

```
console.log(`Operating system runs inside ${system}.`);
```

Output Operating system runs inside MacBook.

2. Default Parameters

What is it? Fallback values sets inside function arguments if calling values are undefined.

Simple example

javascript

```
function bill(cost, tax = 0.05) {
```

```
    return cost + (cost * tax);
```

```
}
```

```
console.log(bill(100)); // uses 5% tax
```

Output 105

3. Optional Chaining (?.)

What is it? Queries deep nested object properties keys safely without throwing `TypeError` reference crashes. Returns `undefined` if lookup fails.

Simple example

javascript

```
let response = { profile: null };  
console.log(response.profile?.name); // returns undefined safely
```

Output undefined

4. Nullish Coalescing (??)

What is it? Operator checks variable status strictly only against **null** and **undefined** states fallback assignment checks.

Comparison: ?? vs || checks all falsy value checks (0, "", false triggers fallback). ?? preserves 0 or "" as valid inputs.

Simple example

javascript

```
let likes = 0;  
console.log(likes || 10); // 10 (incorrect fallback because 0 is falsy)  
console.log(likes ?? 10); // 0 (correct: preserves 0 since it is not nullish)
```

Output 10 0

QUICK REVISION SUMMARY

- **Array destructuring:** Unpacks arrays index levels using [a, , c] brackets.
 - **Object destructuring:** Unpacks keys using curly brackets, renames using colon: { key: aliasName }.
 - **Spread vs Rest:** Spread unpacks and expands iterables; Rest parameter gathers multiple arguments into an array.
 - **Modern ES6 utilities:**
 - Backticks handle template literals.
 - ?. evaluates missing objects safely without throwing TypeError crash.
 - ?? applies fallbacks strictly for null and undefined checks.
-

IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Destructuring Assignment:** Syntactic expression unpacking values from array iterables or object keys into distinct variables.
 2. **Spread Operator:** Syntactic helper expanding collection elements into separate individual values.
 3. **Rest Parameters:** Function parameter declaration syntax grouping variable arguments inside a single array.
 4. **Template Interpolation:** Injecting variable expressions directly inside backtick strings using `${expression}` templates.
 5. **Nullish Coalescing:** Comparison operator verifying parameters strictly against nullish null or undefined states.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Difference between Rest and Spread operators?

Ans: Spread operator expands iterables into separate values (used during assignments or array merges). Rest parameter collects multiple arguments into a single array variable (used in function definitions).

Q2. How does `??` differ from `||` when checking falsy values?

Ans: `||` evaluates against all falsy values (like 0, "", false), triggering fallback. `??` checks strictly only for null and undefined, preserving valid falsy values like 0 and "".

Q3. Predict output:

javascript

```
let [x, y = 5] = [10];
```

```
console.log(x, y);
```

Ans: 10 5. x receives 10, while y defaults to 5.

Q4. Predict output:

javascript

```
const user = { name: "Ravi", role: "admin" };
```

```
const { name: userName } = user;
```

```
console.log(userName);
```

```
console.log(name);
```

Ans: "Ravi" ReferenceError: name is not defined. (Because name was aliased to userName).

Q5. What happens when you use optional chaining on a null reference?

Ans: It stops evaluation immediately and returns undefined instead of throwing a TypeError crash.

Q6. How do you swap variables using array destructuring?

Ans: [a, b] = [b, a]; swaps values without using a temporary variable.

Q7. What does the Rest parameter inside a function argument list look like?

Ans: It looks like function sum(...nums). It collects all extra inputs as an array.

Q8. Predict output:

```
javascript
```

```
let a = [1, 2];
```

```
let b = [3, 4];
```

```
console.log([...a, ...b]);
```

Ans: [1, 2, 3, 4].

Q9. Can we write Rest parameter at the beginning of function arguments?

Ans: No. Rest parameter must always be the last parameter in the function declaration.

Q10. Predict output:

```
javascript
```

```
let obj = { x: 1, y: 2 };
```

```
let { z = 10 } = obj;
```

```
console.log(z);
```

Ans: 10.

Q11. Predict output:

```
javascript
```

```
console.log("" ?? "default");
```

Ans: "" (empty string). Empty string is a valid falsy value, so ?? preserves it.

Q12. Predict output:

javascript

```
console.log("" || "default");
```

Ans: "default". Empty string is falsy, triggering OR fallback.

Q13. Predict output:

javascript

```
let list = [1, 2, 3];
```

```
let [x, ...y] = list;
```

```
console.log(y);
```

Ans: [2, 3]. (Rest operator collects remaining items).

Q14. What are backticks (`) used for in JS?

Ans: Defining template literal strings that support multi-line strings and interpolation.

Q15. Predict output:

javascript

```
let price = 100;
```

```
console.log(`Price is: ${price + 10}`);
```

Ans: "Price is: 110".

Q16. Predict output:

javascript

```
function test(x = 10) {
```

```
    return x;
```

```
}
```

```
console.log(test(undefined));
```

```
console.log(test(null));
```

Ans: 10 (undefined triggers default parameters). null (null is passed as a valid value).

Q17. Predict output:

javascript

```
let user = { info: null };
```

```
console.log(user.info?.age);
```


Ans: undefined.

Q18. How do you destructure properties in a for...of loop?

Ans: By defining the destructuring pattern directly inside the loop declaration: for (let { name } of users).

Q19. Predict output:

javascript

```
let [,z] = [1, 2, 3];
```

```
console.log(z);
```

Ans: 3. Commas skip the first two elements.

Q20. Predict output:

javascript

```
let a = { a: 1 };
```

```
let b = { b: 2 };
```

```
console.log({ ...a, ...b });
```

Ans: { a: 1, b: 2 }.

PRACTICE QUESTIONS

Question 1: Swap variables

- **Question:** Swap values of a = 5 and b = 10 using destructuring.
- **Solution:**

javascript

```
let a = 5;
```

```
let b = 10;
```

```
[a, b] = [b, a];
```

```
console.log(`a: ${a}, b: ${b}`); // a: 10, b: 5
```

- **Explanation:** Swaps values in-place by unpacking values from a transient array.

Question 2: Rename Destructured Keys

- **Question:** Extract role from object { name: "Ravi", role: "Dev" } and rename it to jobTitle.
- **Solution:**

javascript

```
let profile = { name: "Ravi", role: "Dev" };
```

```
const { role: jobTitle } = profile;
```

```
console.log("Job Title:", jobTitle); // Dev
```

- **Explanation:** Uses renaming syntax role: jobTitle to map key value to a new variable.

Question 3: Safely Access Nested Profile

- **Question:** Query zip from { settings: null } using optional chaining and nullish coalescing to fall back to "000000".
- **Solution:**

javascript

```
let data = { settings: null };
```

```
let zipCode = data.settings?.address?.zip ?? "000000";
```

```
console.log("Zip Code:", zipCode); // 000000
```

- **Explanation:** Optional chaining returns undefined safely, then nullish coalescing applies the default fallback.

9:44 PM